action in our application.

This is going too far. We've just limited access to the store itself to administrators. That's not good.

We could go back and change things so that we mark only those methods that specifically need authorization. Such an approach, called *denylisting*, is prone to errors of omission. A much better approach is to *allowlist*—list methods or controllers for which authorization is *not* required. We do this by inserting a `skip_before_action` call within the `StoreController`:

rails7/depot_r/app/controllers/store_controller.rb

```
  class StoreController < ApplicationController
»   skip_before_action :authorize
```

And we do it again for the `SessionsController` class:

rails7/depot_r/app/controllers/sessions_controller.rb

```
  class SessionsController < ApplicationController
»   skip_before_action :authorize
```

We're not done yet; we need to allow people to create, update, and delete carts:

rails7/depot_r/app/controllers/carts_controller.rb

```
  class CartsController < ApplicationController
»   skip_before_action :authorize, only: %i[ create update destroy ]
```

And we allow them to create line items:

rails7/depot_r/app/controllers/line_items_controller.rb

```
  class LineItemsController < ApplicationController
»   skip_before_action :authorize, only: %i[ create ]
```

We also allow them to create orders (which includes access to the `new` form):

rails7/depot_r/app/controllers/orders_controller.rb